



DApp Integration with Smart Contracts

ALU Digital Asset Protection System

Individual Assignment 2

Student: Mozia Precious Chibundu

Institution: African Leadership University

Submission Date: 17 July 2026

GitHub: <https://github.com/Kodedbykenzie/alu-digital-asset-protection>

Abstract

This project extends two previously developed Solidity smart contracts into a complete decentralised application. The React interface allows ALU stakeholders to connect a Web3 wallet, generate SHA-256 fingerprints of image files inside the browser, register digital assets, verify whether a logo is authentic without connecting a wallet, and view or distribute ALUT ownership shares. The system addresses the gap between technically correct blockchain contracts and a product that students, faculty, administrators and external partners can use through a normal web browser.

1. Introduction

Formative Assessment 1 produced two working contracts: ALUAssetRegistry, an ERC-721 registry for unique logo fingerprints, and ALULogoToken, an ERC-20 token with a fixed supply of 1,000,000 ALUT. Although the contracts successfully stored and managed blockchain data, interaction originally required Hardhat commands and JavaScript scripts. This was a major usability gap because ordinary ALU users should not need to understand terminals, contract addresses, ABIs or hexadecimal transaction data.

The Assignment 2 dApp acts as the front door to the contracts. It converts contract functions into clear forms, buttons, status messages and dashboards. The user interface separates public read-only activities from state-changing transactions. Anyone can verify a logo through a read-only RPC call, while registration and token distribution require a connected wallet and explicit approval.

2. Architecture Overview

The application follows a client-side Web3 architecture. React controls the interface and page navigation. ethers.js creates contract objects from the ABI and deployment address. The Web Crypto API hashes files locally. MetaMask supplies the signer for transactions, while a JSON-RPC provider supports public read-only verification.

User	React dApp	ethers.js / Web Crypto	Wallet or RPC Provider	Smart Contracts
Student, staff, public or administrator	Register, Verify and Dashboard pages	ABI calls and SHA-256 hashing	MetaMask signer or read-only provider	ALUAssetRegistry and ALULogoToken

Figure 1. High-level dApp architecture and communication flow.

2.1 ABI and Contract Addresses

An Application Binary Interface (ABI) is a machine-readable description of the public functions, events, parameters and return values exposed by a smart contract. The frontend needs the ABI to encode calls such as registerAsset() and decode returned values such as asset metadata. It also needs the deployed address because the address identifies the exact contract instance on the selected blockchain network.

The updated deployment script deploys both contracts, registers the official logo, creates three example token holders and writes the current addresses, holder addresses and ABIs into the frontend. This reduces configuration mistakes after restarting the local Hardhat network.

3. Feature Walkthrough

3.1 Home Page and Navigation

The home page introduces the purpose of the platform and provides direct links to registration and verification. A persistent navigation bar appears on every page. It includes a prominent Connect Wallet button, and after connection it displays the shortened address and current ALUT balance.

INSERT ACTUAL SCREENSHOT HERE

Figure 2. Home page with the Connect Wallet button and navigation.

Figure 2. Home page with the Connect Wallet button and navigation.

3.2 Wallet Connection

When the user selects Connect Wallet, the frontend checks whether window.ethereum is available. If MetaMask is not installed, the interface presents a helpful installation message. When available, ethers.js creates a BrowserProvider, requests the account, obtains a signer and loads the user balance from ALULogoToken. The dApp also listens for account and chain changes and refreshes its state.

A provider supports reading blockchain state. A signer represents an account that can authorise a transaction. Read-only calls do not require a signature because they do not change the blockchain. Transactions require a signature and gas because they modify shared state. The wallet signs on the user's behalf because the private key belongs to the user and should never be held by the application.

INSERT ACTUAL SCREENSHOT HERE

Figure 3. Connected MetaMask account showing the shortened address and ALUT balance.

Figure 3. Connected MetaMask account showing the shortened address and ALUT balance.

3.3 File Upload and In-Browser Hashing

The upload component accepts common image formats and displays a preview. The browser reads the selected file as an ArrayBuffer and passes its bytes to crypto.subtle.digest("SHA-256", fileBuffer). The resulting 32-byte digest is converted into a 64-character hexadecimal value and prefixed with 0x, producing the bytes32 format required by Solidity.

Hashing in the browser improves privacy and integrity. The complete logo file does not need to leave the user's device, and no server can accidentally alter the bytes before hashing. Only the fixed-length fingerprint and descriptive metadata are sent to the smart contract.

INSERT ACTUAL SCREENSHOT HERE

Figure 4. Uploaded image preview and generated SHA-256 bytes32 fingerprint.

Figure 4. Uploaded image preview and generated SHA-256 bytes32 fingerprint.

3.4 Asset Registration

The registration page pre-fills the content hash generated by the upload component. The user supplies an asset name and confirms the MIME file type. Before sending the transaction, the frontend checks the public registeredContentHashes mapping. The smart contract performs the same duplicate check, ensuring that frontend manipulation cannot bypass the rule.

When the user submits, MetaMask displays the target contract, function transaction and estimated gas information. Approval means that the user agrees to sign and broadcast that exact state-changing operation. After confirmation, the frontend parses the AssetRegistered event from the receipt and displays the new token ID. If an identical hash already exists, the contract reverts and the interface presents a readable duplicate warning.

INSERT ACTUAL SCREENSHOT HERE

Figure 5. Successful asset registration showing the new token ID and transaction hash.

Figure 5. Successful asset registration showing the new token ID and transaction hash.

INSERT ACTUAL SCREENSHOT HERE

Figure 6. Duplicate registration warning for a previously registered fingerprint.

Figure 6. Duplicate registration warning for a previously registered fingerprint.

3.5 Public Logo Verification

The verification page supports both file upload and direct hash input. It does not require a wallet because verifyLogoIntegrity() is a view function. The dApp uses a read-only JSON-RPC provider to call the function and compare the supplied fingerprint with the fingerprint stored for token ID 1. An off-chain view call reads state without creating a blockchain transaction, so the user pays no gas.

When the hashes match, the page presents a green confirmation and loads the registered asset name, file type, registering wallet and timestamp. When they differ, it presents a red warning in plain language. A real-world user could be an ALU partner preparing an event poster. Before publishing, the partner could upload a logo received by email and confirm that it has not been modified. Trust comes from comparing the exact file fingerprint against an immutable contract record rather than trusting the file name or visual appearance.

INSERT ACTUAL SCREENSHOT HERE

Figure 7. Green authentic-logo verification result with registered metadata.

Figure 7. Green authentic-logo verification result with registered metadata.

INSERT ACTUAL SCREENSHOT HERE

Figure 8. Red warning after verifying a modified or incorrect logo.

Figure 8. Red warning after verifying a modified or incorrect logo.

3.6 Token Distribution Dashboard

The dashboard reads the fixed total supply, connected wallet balance, connected wallet ownership percentage and balances of three example stakeholder addresses. The deployment script initially allocates sample shares to a student representative, the communications office and a faculty representative so that the ownership table contains meaningful data.

The distribution form is only displayed when the connected address equals owner(). The frontend validates the recipient address, requires an amount greater than zero and checks the owner balance. However, the true security control remains the onlyOwner modifier in Solidity. If a non-owner calls distributeShares() directly, Ownable reverts the transaction before the transfer executes.

An example practical allocation would be ALUT assigned to the ALU communications office to represent its authorised stewardship of visual identity. In the current system, tokens demonstrate fractional allocation but do not automatically create legal rights. To support voting proportional to holdings, the contract could use OpenZeppelin ERC20Votes, checkpoints, delegation and governance proposal logic, or integrate with a Governor contract.

INSERT ACTUAL SCREENSHOT HERE

Figure 9. Token dashboard with total supply, connected balance and example holders.

Figure 9. Token dashboard with total supply, connected balance and example holders.

INSERT ACTUAL SCREENSHOT HERE

Figure 10. Owner-only share distribution form and confirmed transfer.

Figure 10. Owner-only share distribution form and confirmed transfer.

4. Wallet and Web3 Integration

The frontend maintains wallet state through a React context. connectWallet() requests an account and records the chain ID. It then reads balanceOf(), decimals() and owner() from ALULogoToken. The navigation component consumes this context so the wallet status is consistent on every page.

- BrowserProvider connects ethers.js to the wallet injected into the browser.
- JsonRpcProvider supports public calls without requesting a wallet account.
- Contract instances combine an address, ABI and provider or signer.
- accountsChanged and chainChanged events refresh the interface when the user changes MetaMask state.
- The dApp can request a switch to Hardhat chain ID 31337 and can propose the local network configuration.

5. Verification and Trust Model

A SHA-256 digest changes when even one byte of the image changes. Consequently, a visually subtle edit, recompression or embedded metadata change normally creates a different fingerprint. The smart contract does not decide whether an image looks similar; it performs an exact integrity comparison. This design is suitable for proving whether a file is byte-for-byte identical to the registered official copy.

The public page is trustworthy to the extent that the user trusts the deployed contract address, the blockchain network and the original registration process. The interface exposes the registered metadata and uses a deterministic comparison. For production deployment, ALU should publish the verified contract address through an official institutional channel and deploy to a stable public network or appropriate permissioned blockchain.

6. Automated Test Results

The original eight tests were preserved and five additional dApp integration tests were added, producing thirteen tests in total. The new tests verify the values and transformations used by the interface rather than only the basic contract behaviour.

1. The frontend reads `totalSupply()` and formats the result as 1,000,000.
2. The hashing logic returns the correct SHA-256 bytes32 fingerprint for `alu-logo.png`.
3. The authentic hash produces a success view model.
4. An incorrect hash produces a verification-failure view model.
5. A confirmed `distributeShares()` transaction increases the recipient balance by the correct amount.

INSERT ACTUAL SCREENSHOT HERE

Figure 11. Hardhat terminal output showing all thirteen tests passing.

Figure 11. Hardhat terminal output showing all thirteen tests passing.

7. Security, Design Decisions and Limitations

- No image file is uploaded to an application server; hashing occurs inside the browser.
- The smart contract rejects empty and duplicate content hashes.
- The owner-only token function is protected at contract level, not only by hiding a frontend form.
- No wallet private key is stored in the codebase or application state.
- The default local deployment is appropriate for development and demonstration but cannot serve remote public users.
- Restarting the Hardhat node removes local state, so the contracts must be redeployed and addresses regenerated.
- `ownershipPercentage()` returns whole-number values, meaning holdings below one percent appear as zero percent.
- The current token contract does not implement voting, legal shareholder rights or transfer restrictions.

8. Conclusion

This assignment demonstrated how a smart contract becomes a usable product through frontend integration. The project connected React, ethers.js, MetaMask, the Web Crypto API and Hardhat into one workflow. The most important learning outcome was understanding the distinction between public blockchain reads and signed state-changing transactions. The browser can independently calculate a file fingerprint and verify it for free, while ownership changes remain controlled by wallet signatures and smart-contract access rules.

With more time, the system could be deployed to Sepolia, support WalletConnect, index all historical holders from Transfer events, add role-based ALU administration, implement ERC20Votes governance and publish the frontend through a permanent hosting service. These improvements would move the prototype from a local academic demonstration toward a publicly accessible institutional asset-verification platform.

9. Repository and Video Demonstration

GitHub repository: <https://github.com/Kodedbykenzie/alu-digital-asset-protection>

Deployed website URL: [INSERT FINAL WEBSITE URL]

Video demonstration URL: [INSERT FINAL VIDEO LINK]

References

Ethereum Foundation. Ethereum development networks and smart contract concepts.

Hardhat. Hardhat development environment documentation.

OpenZeppelin. Contracts documentation for ERC721, ERC20 and Ownable.

ethers.js. Version 6 documentation for providers, signers and contracts.

Mozilla Developer Network. Web Crypto API and SubtleCrypto.digest documentation.

React and Vite documentation for frontend development.